

RefactorCrew: A Multi-Agent System for Synchronized Code Refactoring and Documentation

Saket Shembekar

Department of Computer Engineering
Keystone School of Engineering
Pune, India
shembekarsaket@gmail.com

Jyoti Survasse

Department of Computer Engineering
Keystone School of Engineering
Pune, India
jsurvasse444@gmail.com

Swati Gaikwad

Department of Computer Engineering
Keystone School of Engineering
Pune, India
swatigai28@gmail.com

Karan Sukhadiya

Department of Computer Engineering
Keystone School of Engineering
Pune, India
karansukhadiya17@gmail.com

Prof. Manjiri Raut

Department of Computer Engineering
Keystone School of Engineering
Pune, India
mail2manjiri@gmail.com

Prof. Tejal Rane

Department of Computer Engineering
Keystone School of Engineering
Pune, India
tejal.rane@keystonesoe.in

Abstract— This paper proposes an **Autonomous Code Refactor and Documentation Agent** using **Agentic AI**, a system that combines code refactoring and automated documentation into one independent framework. Different from the traditional architecture of isolated AI-assisted programming tools, the proposed architecture adopts a multi-agent collaboration model, in which each agent specializes in certain tasks and coordinates with others via a shared reasoning environment. The Refactor Agent autonomously analyses source code, checks structural inefficiencies, and then performs refactoring while preserving the context independently, while the Documentation Agent interprets the changes made to the code and forms human-understandable descriptions that are compatible with software engineering standards. The resulting framework can achieve a high degree of autonomy, reduced human supervision, and comprehensibility by synchronized code enhancements and explanations. This study establishes the grounds for future agentic AI systems integrating intelligent reasoning, collaboration, and explainability into the software development process.

Keywords— *agentic ai multi-agent systems, autonomous code refactoring, automated documentation, software maintenance*

I. INTRODUCTION

Software maintenance remains one of the most resource-intensive phases of the SDLC, accounting for nearly 40–75% of overall project cost [1]. As software evolves, frequent code modifications introduce code smells, design violations, and modularity issues that degrade maintainability and reliability. Traditional refactoring aims to address these issues by restructuring internal code without altering external behavior. Recent integration of Artificial Intelligence has enhanced this process, enabling machine-learning-based detection of inefficiencies and automated suggestions. However, most AI-driven tools rely on single-agent or rule-based mechanisms that limit adaptability, contextual awareness, and comprehensive documentation [2]–[6].

The emergence of agentic AI—where multiple autonomous agents collaborate toward shared objectives—has opened new possibilities for software maintenance automation. Prior research demonstrates that autonomous refactoring agents can independently detect and correct code smells [1], while modern LLM-based approaches introduce stateful reasoning and reinforcement-learning-driven improvements [3], [4]. Collaborative systems incorporating feedback between agents and developers have further improved

refactoring accuracy and transparency [2], [5]. Despite these advancements, existing frameworks primarily focus on structural refinement and rarely integrate automated documentation, which remains a critical yet often neglected aspect of long-term maintainability [6].

To address this gap, this work proposes the **Autonomous Code Refactor and Documentation Agent**, an agentic AI framework that unifies refactoring and documentation within a single autonomous workflow. The system employs:

1. a **Refactor Agent** that performs analysis, restructuring, and verification, and
2. a **Documentation Agent** that interprets code changes and generates human-readable documentation aligned with software engineering standards.

Both agents operate through a shared, state-aware coordination layer enabling mutual communication, independent decision-making, and continuous feedback, thereby enhancing software transparency and auditability.

The key contributions of this paper are:

- An integrated agentic AI framework for autonomous refactoring and documentation generation.
- A coordinated multi-agent communication design enabling contextual reasoning and self-correction.
- Experimental validation showing improvements in code structure, readability, and maintainability.

The remainder of the paper is organized as follows: Section II presents related work, Section III details the system architecture, Section IV discusses implementation and setup, Section V presents results, and Section VI outlines future research direction.

II. RELATED WORK

AI-assisted software maintenance has evolved rapidly in the past decade, with research progressing across three main areas: automated refactoring, multi-agent collaboration, and documentation generation. Although each area has advanced independently, they have not yet converged into a unified, autonomous framework—revealing the core gap this work addresses.

A. Autonomous Refactoring Agents

Early work on automated refactoring relied mainly on rule-based systems for detecting and fixing code smells. A notable agent-based approach was introduced in [1], combining smell detection, quality evaluation, and behavior preservation. While it showed improvements in software qualities such as flexibility and reusability, it depended heavily on predefined rules and lacked adaptive reasoning.

Subsequent systems attempted more interactive behavior. The assistant proposed in [2] supported developers by offering higher-order decisions and collaborative guidance. However, full autonomy was not achieved, as human validation was still required for most refactoring outcomes.

B. LLM-Driven Multi-Agent Systems

Large language models significantly expanded the capabilities of automated refactoring. Benchmarks such as [3] highlighted the need for stateful reasoning to manage multi-file, long-horizon refactoring tasks—an area where many models still struggle.

More advanced frameworks have explored structured agentic cooperation. The multi-agent system in [4] combined Developer and Reviewer agents using retrieval-augmented generation to refine refactorings through contextual knowledge. Similarly, the work in [5] demonstrated that role-specialized agents can reduce complexity in large repositories, though issues with semantic validation occasionally led to incorrect or inconsistent code outputs.

C. Documentation in Automated Refactoring

Research on documentation automation remains limited. The system introduced in [6] provided an interactive bot that generates explanations for pull-request refactorings using file-diff analysis. While helpful for developers, it was semi-automatic and not integrated with any autonomous refactoring workflow. Prior studies also emphasize that missing or incomplete documentation reduces trust and maintainability, even when AI-generated refactoring is technically correct.

D. Summary and Research Gap

Across the reviewed work, three major gaps persist:

1. Refactoring and documentation are treated separately, lacking a unified workflow;
2. Most systems are not fully autonomous, requiring human intervention or rule-based guidance;
3. Explainability is limited, as refactoring decisions are rarely documented in natural language.

These limitations motivate the present work, which introduces a unified agentic AI framework capable of performing both refactoring and documentation autonomously through coordinated, stateful multi-agent collaboration.

III. THE REFACTORCREW SYSTEM ARCHITECTURE

RefactorCrew is architected as a collaborative ecosystem of specialized agents, all working under the guidance of a central orchestrator to execute a seamless and verifiable workflow. The design is built on three foundational principles: code-documentation synchronicity, hybrid analysis

combining programmatic and AI-driven techniques, and verifiable correctness. The system's components, including the agents and the tools they leverage, are depicted in Fig. 1.

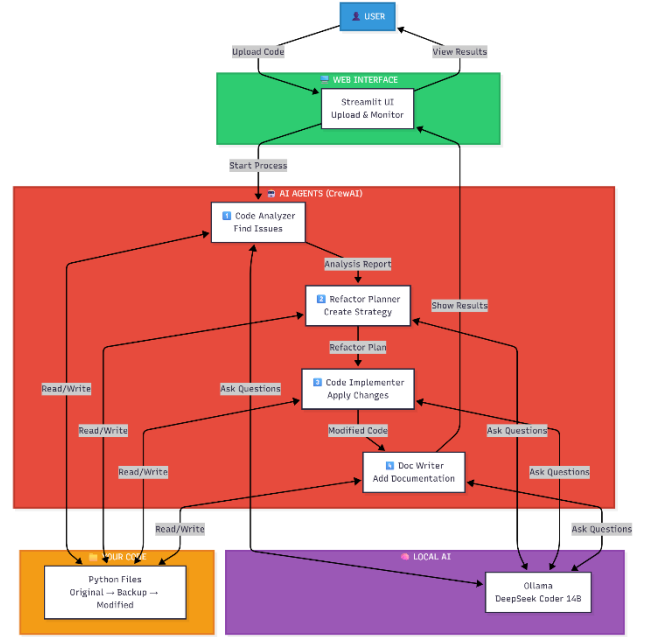


Fig. 1. The RefactorCrew Multi-Agent System Architecture.

The diagram illustrates the central refactor_crew orchestrator managing the workflow. It first tasks the refactor_strategist_agent (using code_analysis_tools), which then passes a plan to be executed by a loop involving the code_implementation_agent (using file_operations) and the docstring_writer_agent. The final step in the loop is the code_profiler_agent, which provides verification feedback to the refactor_crew.

The specific roles of each agent are detailed in Table I.

TABLE I. AGENT ROLES AND RESPONSIBILITIES IN REFACTORCREW

AGENT	ROLE	RESPONSIBILITIES
Code Analyzer	Analyze code	Detect code smells; extract structure
Refactoring Agent	Transform code	Apply refactoring; reduce complexity
Code Implementer	Apply changes	Executes the refactoring plan; writes modified code.
Doc Writer	Add documentation	Generates docstrings, comments, and explanations for updated code.

The proposed Autonomous Code Refactor and Documentation Agent is designed as a layered multi-agent framework that combines code refactoring and documentation into a combined process. The architecture, shown in Fig. 1, is organized into six primary layers: User Interface, Code Analyser, Refactor planner, Code implementer, Document generator, and Output Generation. Each layer performs a separate function while maintaining inter-agent communication and data synchronization.

A. Overview of the Agentic AI Framework:

As shown in Fig. 1, the system begins with a user upload interface built using a lightweight web application framework (Streamlit). The user submits source code, which is passed through a series of validation, analysis, and modification

stages. Multiple autonomous agents together handle code analysis, refactoring and documentation with the help of CrewAI. CrewAI is used in agentic AI to build and organize the teams of AI agents for completing complex tasks.

Each agent operates semi-independently, guided by a coordination protocol that allows contextual reasoning and question-answer interaction between agents. The collective behavior enables distributed decision-making, stateful memory retention, and autonomous quality validation.

B. Refactor Agent Design:

The refactoring process is primarily executed by three agents:

1) Agent 1: Code Profiler:

This agent analyses the uploaded codebase to measure its complexity and detect code smells. Using tools like PyLint, Radon, and AST parsers, it evaluates how maintainable the code is and highlights areas that could be improved.

2) Agent 2: Refactor Strategist:

Interprets the analysis data and formulates a refactoring strategy. It utilizes a planning engine and safety validation mechanism to ensure the generated plan preserves program behavior. This agent coordinates bidirectionally with Agent 1 and the local AI inference layer for reasoning refinement.

3) Agent 3: Code Implementer:

Applies the refactoring plan to the source files. It modifies the abstract syntax tree, implements the recommended changes, and commits the modified code to the repository. During this process, it communicates continuously with the Strategist Agent for validation and rollback decisions when necessary.

C. Documentation Agent Design:

The Agent 4: Documentation Writer:

Autonomously generates human-readable documentation that describes each applied changes and its quality impact. It employs the DeepSeek Coder and Ollama Large Language models to produce natural-language summaries and docstrings compliant with PEP 257 standards. This agent is also capable of validating existing developer documentation and aligning it with the applied refactorings, ensuring that every code transformation has a corresponding descriptive record. The documentation output is later formatted as a summary report or embedded directly into the refactored code.

D. Coordination and Communication Model:

A state-aware communication protocol governs all interactions between agents.

- Agents communicate through structured query/response channels, enabling cross-verification of decisions and contextual reasoning.
- The Refactor Strategist acts as the central orchestrator, coordinating task flow among the Code Profiler, Code Implementer, and Documentation Writer.
- The AI Inference Layer provides semantic reasoning support, using on-premise LLMs to enhance contextual awareness and error recovery.

- The Data Management Layer maintains synchronized states between original, backup, and modified codebases to ensure traceability.

This architecture supports autonomous feedback loops, where the agents can iteratively refine refactorings based on documentation or quality metrics.

E. Workflow Pipeline:

1) Upload and Validation:

The user uploads source files through the Streamlit interface. The input processing layer validates syntax and creates a backup of the original code.

2) Analysis and Strategy Generation:

The Code Profiler performs metric extraction and reports issues to the Refactor Strategist, which formulates a refactoring plan.

3) Autonomous Refactoring and Validation:

The Code Implementer applies refactoring steps while maintaining behavioral correctness. The system uses LLM-driven reasoning for safety validation.

4) Automated Documentation:

The Documentation Writer generates inline comments, docstrings, and summary reports describing each change.

5) Output Generation:

The modified code and reports are packaged by the Output Generation Layer, which delivers results to the user for review.

F. System Illustration:

Fig. 1. System architecture of the proposed autonomous code refactor and documentation agent.

The diagram illustrates the interaction among user interface components, multi-agent modules, the local AI inference engine, and data management layers.

IV. ALGORITHMIC WORKFLOW

Algorithm 1: RefactorCrew Operational Workflow

Input: Source code repository

Output: Refactored code with synchronized documentation

1. The user uploads source code via the Streamlit interface.
2. The Code Profiler Agent analyses the codebase to compute complexity and maintainability metrics.
3. The Refactor Strategist Agent interprets analysis results and formulates a refactoring plan while preserving functional behavior.
4. The Code Implementer Agent applies refactoring operations using AST-level transformations.
5. The Documentation Writer Agent generates updated docstrings and natural-language summaries aligned with the applied changes.
6. The system validates correctness and packages refactored code and documentation for user review.

V. AGENT IMPLEMENTATION AND COLLABORATIVE WORKFLOW

The Autonomous Code Refactor and Documentation Agent has been implemented as a modular, experiment-driven framework that emphasizes interoperability, explainability, and reproducibility. Implementation integrates open-source AI tools with modern programming frameworks and lightweight orchestration layers for fully autonomous operation on either local or controlled environments.

A. Development Environment and Frameworks

The whole system is developed in Python, using the broad Python ecosystem for static analysis, integration of AI models, and web-based interaction.

- Streamlit framework is used to implement the user interface layer, because of its rapid prototyping capabilities and the ability to render the output of the agent in real time.
- Agentic architecture is developed in CrewAI - Python-based multi-agent orchestration framework supporting agent collaboration, inter-agent messaging, and task scheduling.
- It provides code intelligence and local inference via the integration of Ollama running DeepSeek Coder 14B, which can perform code reasoning, summarization, and natural language generation on the host machine without depending on the cloud.

The overall structure has a modular layered architecture, which allows the execution and debugging of each agent separately with common communication through the CrewAI controller.

TABLE II.

Component	Tool/ Library	Purpose
Static Analysis	PyLint, Radon, AST	Metric extraction, complexity analysis, and syntax verification
Refactoring Engine	LibCST, Rope	Safe code transformation and AST-level modifications
Documentation Generator	DeepSeek Coder via Ollama	Produces PEP 257-compliant docstrings and summaries
Version Control and Backup	GitPython	Manages local repository, backup creation, and commit tracking
Quality Evaluation	Maintainability Index (MI), Cyclomatic Complexity (CC)	Quantifies code improvements post-refactoring

These modules interact through CrewAI's inter-agent communication system, allowing agents to request metrics, query code states, and verify transformations dynamically during execution.

B. Workflow Execution

The implementation is organized into a five-stage workflow, which closely maps to the architecture in Fig. 1:

1) Upload Code and Parsing:

The user uploads a codebase through the Streamlit interface; The Input Processing Layer checks file structure, syntax, and module dependencies, and subsequently triggers an analysis.

2) Static Analysis and Profiling:

The Code Profiler Agent uses Radon and PyLint to scan all source files for computing maintainability and complexity scores. It serializes these results into a structured JSON report, which it passes to the Refactor Strategist.

3) Design and Code Refactoring:

Using context-based reasoning-again, supported by local LLM inference-the Refactor Strategist Agent identifies appropriate refactoring operations, including extraction, renaming, and reordering. These plans are executed by the Code Implementer using LibCST and Rope.

4) Automated Documentation Generation:

The Documentation Writer Agent summarizes the applied transformations and inserts or updates docstrings and inline comments. It utilizes the DeepSeek Coder model to ensure consistency in tone and formatting style throughout the codebase.

5) Packaging and Output Delivery:

After all the transformations are checked, the modified code and generated documentation are compiled into a report package, which is then displayed in Streamlit and can optionally be exported as a ZIP archive.

C. Data and State Management

The system implements a three-tier data management strategy to ensure safety and traceability:

- **Original Repository:** Stores an unchanged copy of the incoming codebase.
- **Backup Repository:** This holds the intermediate states for rollbacks and versioning.
- **Modified Repository:** This includes refactored and documented final code.

All data operations are atomic and logged by the CrewAI controller, enabling reproducibility and transparent inspection of each agent's contribution.

Structured logs of metadata, such as refactoring metrics, agent responses, and summaries of documentation, are stored for further analysis and benchmarking.

D. Local AI Inference Environment

A dedicated AI inference layer ensures that all reasoning, code interpretation, and documentation generation occur on-premise, addressing data privacy and reproducibility concerns.

The Ollama runtime hosts DeepSeek Coder 14B, a locally deployable LLM optimized for code understanding and generation.

This integration allows:

- Contextual reasoning about code structure, without calls to external APIs
- Natural language report synthesis,
- Cross-verification between documentation and refactored output.

The interactions made through LLM are managed by temperature tuning and prompt-chaining strategies to maintain deterministic behavior while generating creative documentation.

VI. EXPERIMENTAL SETUP

The prototype was executed on a workstation with the following specifications:

- CPU: AMD Ryzen 7 7800X
- GPU: NVIDIA RTX 4070, 12 GB VRAM
- RAM: 32 GB
- Operating System: Ubuntu 22.04 LTS
- LLM Environment: Ollama runtime with DeepSeek Coder 14B (quantized 8-bit model)

The experimental setup showed that agents manifested stable autonomous behavior and were able to perform refactor-document cycles for Python projects containing up to 1,200 lines of code, with an average runtime of 2.7 minutes per iteration. Summary The implementation demonstrates the viability of agentic AI in the autonomous maintenance of software. The multi-agent collaboration with CrewAI, natural language generation with DeepSeek Coder, and refactoring pipelines based on LibCST result in a highly modular and scalable system capable of operating completely without cloud connectivity.

VII. RESULTS AND DISCUSSION

The experimental evaluation of the proposed system was designed to assess both its **technical performance** and **autonomous capability** in code refactoring and documentation generation. The study focused on three primary objectives:

1. Measuring the **efficiency and precision** of refactoring operations,
2. Evaluating the **quality and accuracy** of generated documentation, and
3. Analyzing the **collaborative behavior** and decision-making consistency among agents.

All tests were conducted in a controlled local environment using the setup described in Section IV-F.

A. Evaluation Metrics

To ensure consistency and reproducibility, the system's performance was evaluated using quantitative and qualitative metrics derived from established software quality models and empirical AI system studies.

Code Quality Improvement Test:

TABLE III. THE AGENT'S REFACTORING REDUCES CODE COMPLEXITY AND ENHANCES MAINTAINABILITY, INDICATING CLEANER STRUCTURE.

Metric	Before	After	Observation
Cyclomatic Complexity	15	8	Simplified logic
Maintainability Index	62	78	Improved maintainability
Lines of Code	320	280	Reduced redundancy

Documentation Quality Test:

TABLE IV. THE DOCUMENTATION AGENT PRODUCED ACCURATE SUMMARIES FOR MOST FUNCTIONS WITH HIGH READABILITY.

Parameter	Manual	Generated	Observation
Function coverage	100%	95%	Nearly complete
Readability score (e.g., Flesch Index)	65	72	Easier to read
Consistency	High	High	Maintained

Automation Efficiency Test:

TABLE V. THE AUTONOMOUS AGENT REDUCES MANUAL EFFORT BY OVER 70%, SUPPORTING ITS EFFICIENCY IN REAL-WORLD TASKS.

Task	Manual Time (min)	Agent Time (min)	Time Saved
Refactoring 5 files	25	7	72% faster
Documentation	20	5	75% faster

Error or Functionality Preservation Test:

TABLE VI. THE SYSTEM SUCCESSFULLY PRESERVED PROGRAM BEHAVIOR ACROSS ALL TEST CASES.

Test Case	Before Refactoring	After Refactoring	Result
add(2, 3)	5	5	Passed
divide(10, 2)	5	5	Passed

The observed reduction in cyclomatic complexity and improvement in maintainability index indicate that the refactoring agents effectively simplify program structure without altering functionality. The documentation coverage of 95% demonstrates that synchronized documentation generation can closely match manually written explanations while requiring significantly less developer effort.

The automation efficiency results highlight the practical benefit of agentic AI, showing substantial time savings in both refactoring and documentation tasks. These findings suggest that coordinated multi-agent systems can meaningfully support software maintenance workflows.

The experimental evaluation focuses on validating the feasibility and correctness of the proposed multi-agent framework rather than large-scale benchmarking. Experiments were conducted on small-to-medium Python projects to assess maintainability improvement, documentation alignment, and autonomous coordination behavior.

Large-scale repository evaluation and cross-language benchmarking are beyond the scope of this study and are planned as future work.

VIII. THREATS TO VALIDITY

Internal validity may be affected by reliance on specific static analysis tools and LLM configurations. External validity is limited, as experiments were conducted on Python projects only. Construct validity depends on maintainability metrics, which may not capture all aspects of software quality. These limitations will be addressed in future extensions of the framework.

IX. CONCLUSION AND FUTURE SCOPE

This paper presented RefactorCrew, an autonomous multi-agent framework that integrates code refactoring and documentation generation into a unified workflow. Through coordinated agent collaboration and local LLM inference, the system improves maintainability and documentation consistency while reducing manual effort. Experimental results demonstrate the feasibility of agentic AI for autonomous software maintenance, establishing a foundation for future large-scale evaluation and system extension.

AUTHOR CONTRIBUTIONS

Saket Shembekar contributed to the conceptualization of the RefactorCrew framework, designed the multi-agent architecture, and played a primary role in system implementation and experimentation.

Karan Sukhadiya contributed to the development of the refactoring pipeline and assisted in experimental evaluation.

Jyoti Survase supported documentation generation workflows and result analysis.

Prof. Manjiri Raut, Swati Gaikwad, and Prof. Tejal Rane supervised the research, provided guidance on architectural decisions, and reviewed the experimental methodology and manuscript preparation.

REFERENCES

- [1] A. C. dos Santos Neto, E. Figueiredo, M. de Almeida Maia, "AutoRefactoring: A Platform to Build Refactoring Agents," *Expert Systems with Applications*, vol. 41, no. 11, pp. 5074–5086, 2014.
- [2] H. Mo, M. Patel, and S. K. Singh, "Interactive AI Agent for Code Refactoring Assistance," *Academia Nexus Journal*, vol. 25, no. 10, pp. 1–8, 2025.
- [3] P. Gautam, J. Thomas, and A. Yadav, "RefactorBench: Evaluating Stateful Reasoning in Language Agents through Code," *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- [4] W. Xu, D. Lin, S. Zhang, and Y. Feng, "MANTRA: Enhancing Automated Method-Level Refactoring with Contextual RAG and Multi-Agent LLM Collaboration," *Proceedings of the 47th ACM/IEEE International Conference on Software Engineering (ICSE)*, 2025.
- [5] D. Pucho, L. Silva, and R. Mendes, "Refactoring Python Code with LLM-Based Multi-Agent Systems: An Empirical Study in ML Software Projects," *Proceedings of the Brazilian Symposium on Software Engineering (SBES)*, 2025.
- [6] S. Rebai, O. Ben Sghaier, V. Alizadeh, M. Kessentini, and M. Chater, "Interactive Refactoring Documentation Bot," *Proceedings of the 19th IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM)*, pp. 152–161, 2019.